

O'REILLY®

Second
Edition

Generative Deep Learning

Teaching Machines to Paint, Write,
Compose, and Play



David Foster
Foreword by Karl Friston

Chapter 7. Energy-Based Models

심교남

2026/01/19

<https://kyonami.github.io/>

개요

- 에너지 기반 모델: 물리 시스템 모델링에서 핵심 아이디어를 차용한 생성 모델 분야
 - 물리 시스템 모델링: 현실의 물리적 시스템(전자 회로, 유체 흐름)을 방정식, 규칙, 그래프 등으로 추상화하여 컴퓨터로 시뮬레이션이나 분석 하기 위해 구조화 하는 것
 - 실수 값의 에너지 함수를 0과 1 사이로 규정하는 함수은 볼츠만 분포를 통해 어떤 이벤트의 확률을 표현할 수 있음
 - 볼츠만 분포: 열 평형 상태의 기체를 설명하는 데 사용되는 공식



소개

- 뛰어난 실력의 달리기 트레이너가 있음
 - 경기 후 특정 선수의 에너지가 얼마나 남았는지 보기만 해도 파악이 가능함
 - 선수의 에너지 레벨이 낮을 수록 더 좋음
 - 엘리트 선수는 모든 것을 쏟아붓기 때문
- 트레이너가 기술을 예리하게 유지하려 함
 - 그래서 정기적으로 엘리트 선수들과 클럽 최고 선수들간 에너지 감지 능력 차이 측정 및 훈련
 - 클럽에서 진짜 엘리트를 찾았단 말을 믿게 하기 위해 둘의 예측 차를 최대한 크게 만듦
- 최고 능력은 평범한 러너를 최고 수준의 러너로 만드는 것
 - 선수의 현재 에너지를 측정하고 다음 경기를 위한 최적의 에너지 조정 값을 찾아냄
 - 조정 후 다시 측정해 이전보다 에너지레벨이 낮아져 더 향상되었는지 확인
 - 세계적 선수와 구별할 수 없을 때 까지 계속 함
- 이는 에너지 기반 모델링의 핵심 아이디어를 잘 설명하는 예시



에너지 기반 모델

- 볼츠만 분포를 사용하여 데이터 생성 분포를 모델링함

- 볼츠만 분포: $p(x) = \frac{e^{-E(x)}}{\int_{\hat{x} \in X} e^{-E(\hat{x})}}$

- x : 샘플
- $E(x)$: 샘플 x 의 에너지 함수(또는 점수)
- 분자 $e^{-E(x)}$: 확률처럼 보이는 점수
 - $E(x)$ 가 작아지면 $-E(x)$ 가 커짐 $\rightarrow$ 분자가 커짐 $\rightarrow$ 에너지가 낮으면 확률이 높아짐
- 분모 $\int_{\hat{x} \in X} e^{-E(\hat{x})}$: 전체를 다 더해서 적분하여 정규화 시킴
 - 모든 x 에 대한 확률의 합이 1이 되게 만들어줌
- 소프트맥스와 유사한 역할



에너지 기반 모델

- 볼츠만 분포의 문제

- 1. 새로운 샘플 생성시에 모델을 어떻게 사용해야 할 지 불명확

- 모델을 사용해 주어진 샘플에 어떤 점수를 출력할 수 있지만
 - 점수가 낮은 샘플을 생성할 방법이 모호함

- 2. 분모에 간단한 문제를 제외하고는 풀기 어려운 적분이 있음

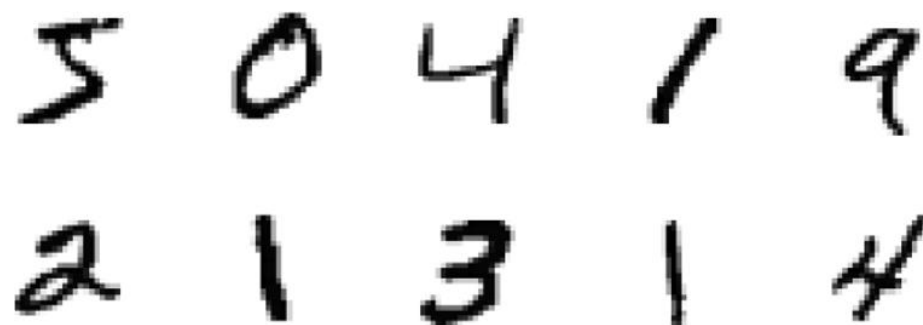
- 이 적분을 계산할 수 없으면 최대 가능도 추정을 통해 모델 학습이 불가함
 - 최대 가능도 추정을 위해선 $p(x)$ 가 유효한 확률 분포여야 함
 - 에너지 기반 모델은 근사 기법을 통해 이 까다로운 분모 계산을 안하는 것
 - 표준 가우스 분포에 적용한 변환이 유효한 확률분포를 출력하는 노멀라이징 플로우와 대조적
 - 노멀라이징 플로우는 8장의 내용



MNIST 데이터셋

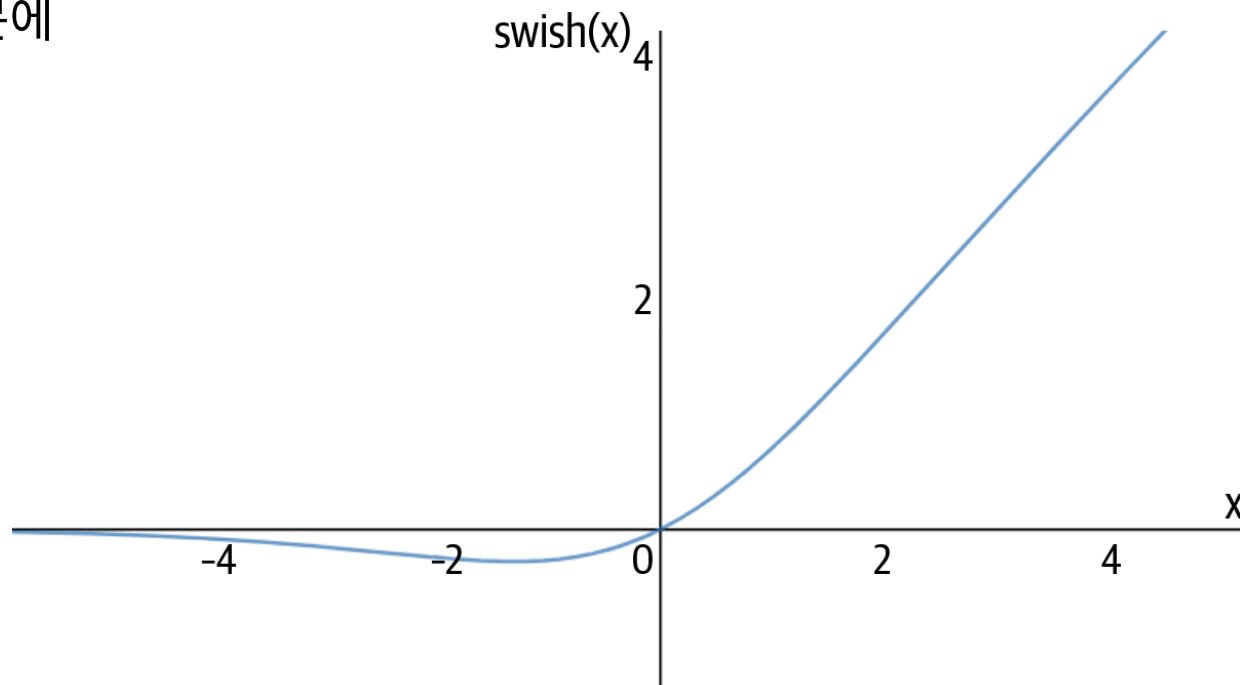
- 간단한 E(x)에 대한 신경망 설계에 MNIST 데이터셋 활용
- 손 글씨 이미지로 구성된 표준 MNIST 데이터셋 활용
- 픽셀 값을 $[-1, 1]$ 범위로 조정하고 32x32 사이즈에 맞춰 패딩

```
1 from tensorflow.keras import datasets
2 (x_train, _), (x_test, _) = datasets.mnist.load_data()
3
4 def preprocess(imgs):
5     imgs = (imgs.astype("float32") - 127.5) / 127.5
6     imgs = np.pad(imgs, ((0,0), (2,2), (2,2)), constant_values= -1.0)
7     imgs = np.expand_dims(imgs, -1)
8     return imgs
9
10 x_train = preprocess(x_train)
11 x_test = preprocess(x_test)
12
13 x_train = tf.data.Dataset.from_tensor_slices(x_train).batch(128)
14 x_test = tf.data.Dataset.from_tensor_slices(x_test).batch(128)
```



에너지 함수

- 에너지 함수 $E_{\theta}(x)$
 - 파라미터가 θ 이고 입력 이미지 x 를 하나의 스칼라 값으로 변환하는 신경망
- 활성화 함수: $swish(x) = x \cdot sigmoid(x) = \frac{x}{e^{-x} + 1}$
 - ReLU와 모양이 비슷하지만 매끄럽기 때문에 Vanishing Gradient 문제를 완화시켜줌



에너지 함수

- 이 신경망은 일련의 Conv2D 층을 쌓아 이미지 크기를 줄이고 채널을 늘림
 - 마지막 층은 선형 활성화를 가진 하나의 완전 연결 유닛
 - $(-\infty, \infty)$ 범위의 값을 출력함

```
ebm_input = layers.Input(shape=(32, 32, 1))
x = layers.Conv2D(
    16, kernel_size=5, strides=2, padding="same", activation = activations.swish
)(ebm_input) # 에너지 함수는 Swish 활성화 함수를 가진 Conv2D 층을 쌓아서 만들
x = layers.Conv2D(
    32, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Conv2D(
    64, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Conv2D(
    64, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation = activations.swish)(x)
ebm_output = layers.Dense(1)(x) # 마지막층은 선형 활성화를 가진 하나의 완전 연결 유닛
model = models.Model(ebm_input, ebm_output)
# 입력 이미지를 하나의 에너지 점숫값으로 변환하는 Keras의 Model 클래스 객체를 만들
```



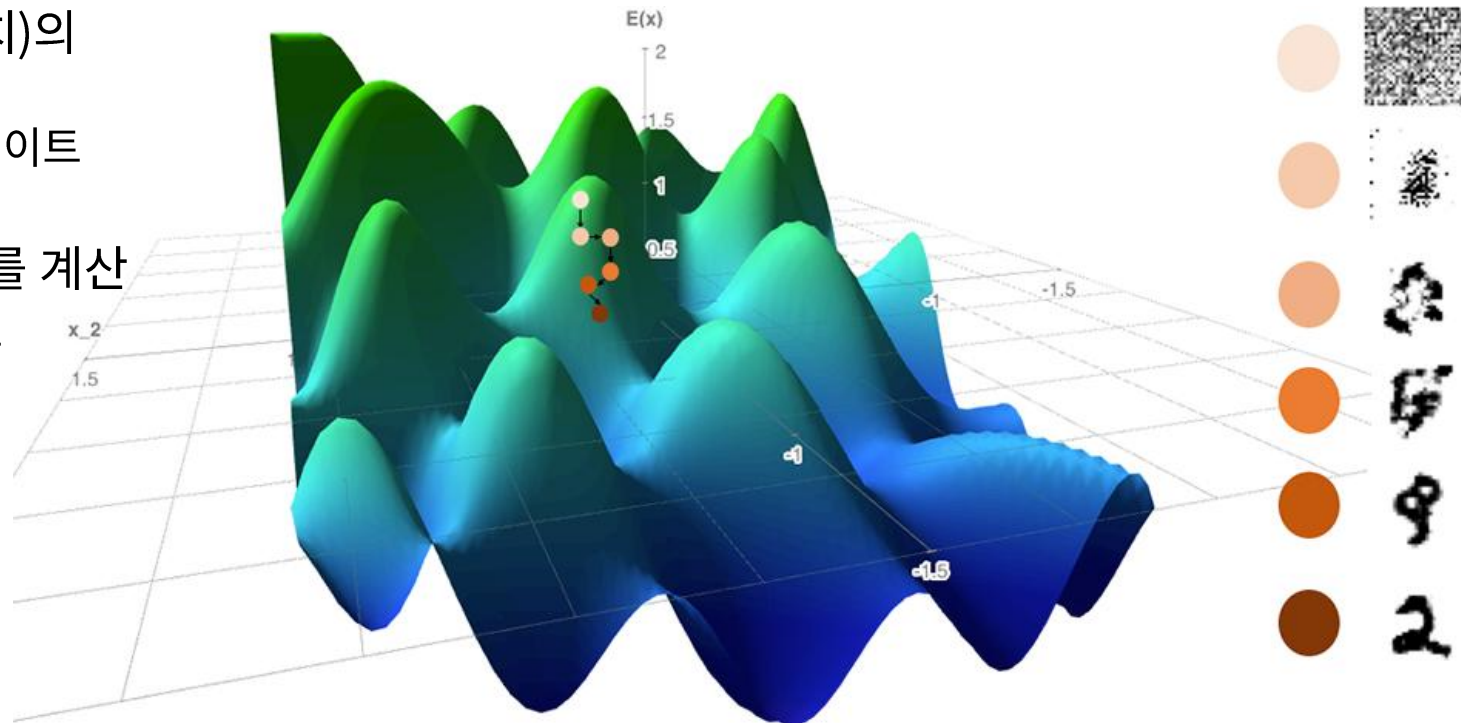
Langevin Dynamics를 통한 샘플링

- 에너지 함수는 입력에 대해 하나의 점수만 출력함
 - 에너지 점수가 낮은 새로운 샘플을 생성하기 위해 Langevin Dynamics 기법이 필요함
- Langevin Dynamics: 입력에 대한 에너지 함수의 그레이디언트를 계산하는 기법
 - 먼저 샘플 공간의 임의의 지점에서 시작하여
계산된 그레이디언트의 반대 방향으로 조금씩 이동하면 에너지 함수가 점점 감소됨
 - 올바르게 훈련된 신경망이면 랜덤한 잡음이 훈련 세트에 있는 유사한 이미지로 변환됨
- 샘플 공간 이동시 소량의 랜덤 잡음이 필요함
 - 그렇지 않으면 Local minima에 빠질 수 있음
 - 이 기법을 확률적 경사 Langevin Dynamics라고 함



Langevin Dynamics를 통한 샘플링

- 확률적 경사 하강법을 3차원으로 시각화 할 수 있음
 - 이 경로는 입력 x 에 대한 에너지 함수 $E(x)$ 의 음의 그래디언트를 따르고 잡음이 있는 내리막길
 - 1024(32x32) 픽셀이 있기 때문에 1024차원 공간을 탐험하는 것이지만 동일한 원리가 적용됨
- 일반적인 경사 하강법과는 차이가 있음
 - 일반적 경사하강법은 파라미터(가중치)의 손실함수의 기울기를 계산
 - 음의 기울기 방향으로 파라미터 업데이트
 - 손실을 최소화
 - 여기서 가중치가 아닌 출력의 기울기를 계산
 - 음의 기울기 방향으로 입력 업데이트
 - 출력(에너지 점수)을 최소화



Langevin Dynamics를 통한 샘플링

- 랑주뱅 동역학 방정식: $x^k = x^{k-1} - \eta \nabla_x E_\theta(x^{k-1}) + \omega$
 - , $x^0 \sim U(-1.1)$
 - $x^0 \sim U(-1, 1)$ 은 $[-1, 1]$ 범위의 균등 분포임
 - η 는 튜닝해야 할 스텝의 크기 (하이퍼파라미터)
 - 스텝이 너무 크면 최소값을 건너뛰고, 너무 작으면 수렴이 느려짐
 - x^k : k번째로 수정된 이미지
 - $\nabla_x E_\theta$: 입력 이미지 픽셀을 조금 바꾸면 얼마나 변하는지 (기울기)
 - $-\eta \nabla_x E$: 에너지를 줄이는 방향을(내리막)으로 η 만큼 이동
 - ω : 가우시안 잡음



Langevin Dynamics를 통한 샘플링

- Langevin 샘플링 함수를 작성한 예제 코드

```
def generate_samples(model, inp_imgs, steps, step_size, noise):  
    for _ in range(steps): # 주어진 스텝 횟수를 반복  
        # 이미지에 소량의 잡음 추가  
        inp_imgs += tf.random.normal(inp_imgs.shape, mean = 0, stddev = noise)  
        inp_imgs = tf.clip_by_value(inp_imgs, -1.0, 1.0)  
        with tf.GradientTape() as tape:  
            tape.watch(inp_imgs)  
            # 이미지를 모델에 통과시켜 에너지 점수를 얻음  
            out_score = -model(inp_imgs)  
        # 입력에 대한 출력의 그레이디언트를 계산  
        grads = tape.gradient(out_score, inp_imgs)  
        grads = tf.clip_by_value(grads, -0.03, 0.03)  
        # 작은 양의 그레이디언트를 입력 이미지에 더함  
        inp_imgs += -step_size * grads  
        inp_imgs = tf.clip_by_value(inp_imgs, -1.0, 1.0)  
    return inp_imgs
```



Contrastive Divergence를 통한 훈련

- 에너지 함수는 확률을 출력하지 않기 때문에 최대 가능도 추정을 적용할 수 없음
 - 출력 점수가 샘플 공간 전체를 더해서 1이 되지 않음
 - 이때 Contrastive Divergence이라는 기법을 적용함
- $\mathcal{L} = -\mathbb{E}_{x \sim data}[\log p_{\theta}(x)]$: 최소화 하려는 값은 음의 로그 가능도
- $p_{\theta}(x)$ 가 에너지 함수 $E_{\theta}(x)$ 를 포함하는 볼츠만 분포의 형태일 경우의 기울기
 - $\nabla_{\theta} \mathcal{L} = \mathbb{E}_{x \sim data}[\nabla_{\theta} E_{\theta}(x)] - \mathbb{E}_{x \sim model}[\nabla_{\theta} E_{\theta}(x)]$
 - 실제 샘플의 에너지 점수는 큰 음으로, 생성된 가짜 샘플의 점수는 큰 양으로 출력되도록 훈련
 - 차이가 가능한 한 커지도록 하여 손실함수로 사용함
- 가짜 샘플의 에너지 점수를 계산하려면 $p_{\theta}(x)$ 에서 정확히 샘플링 되어야 함
 - 하지만 분모 계산이 어려워서 불가능함
 - Langevin 샘플링을 통해 낮은 점수를 가진 샘플 집합 생성이 가능함
 - 무한히 반복하면 완벽해지지만 현실적으로 적은 스텝을 수행함



Contrastive Divergence를 통한 훈련

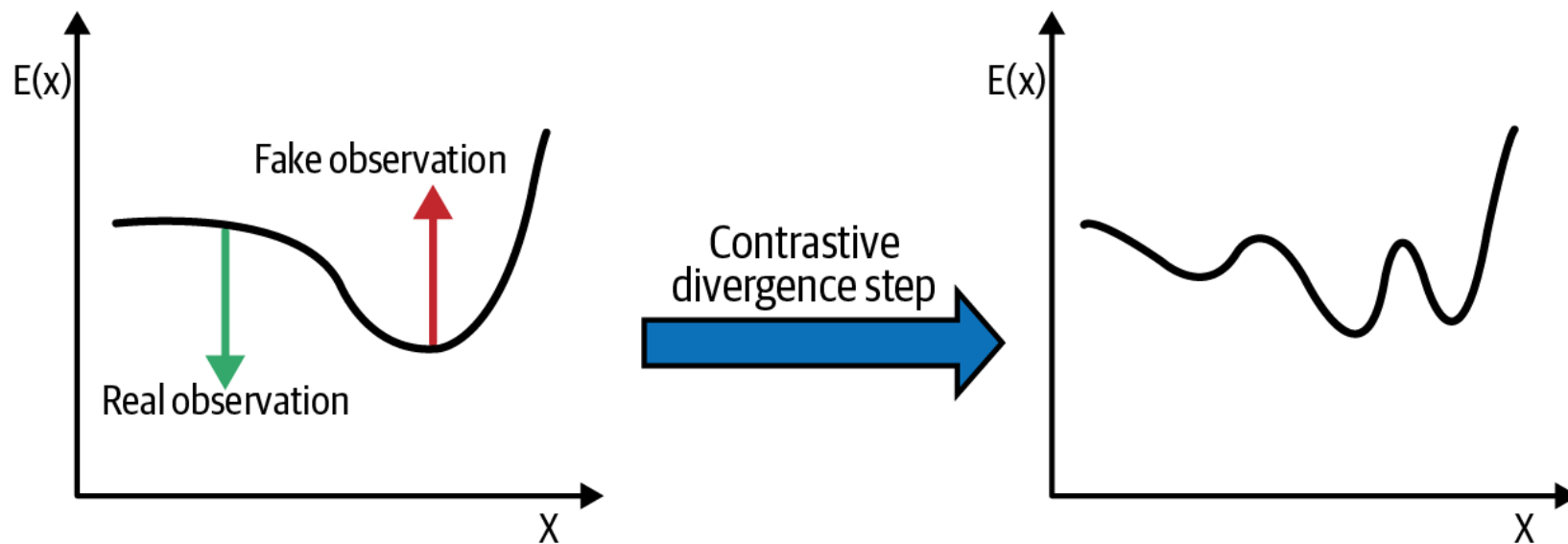
- 이전 반복의 샘플을 버퍼에 저장하여 다음 배치의 시작점으로 노이즈 대신 사용 가능
- 우측은 버퍼의 예제코드

```
class Buffer:
def __init__(self, model):
    super().__init__()
    self.model = model
    self.examples = [
        tf.random.uniform(shape = (1, 32, 32, 1)) * 2 - 1
        for _ in range(128)
    ] # 샘플링 버퍼는 랜덤한 잡음의 배치로 초기화 됨

def sample_new_exmps(self, steps, step_size, noise):
    # 매번 평균적으로 5%가 처음부터 (즉 노이즈로부터) 생성
    n_new = np.random.binomial(128, 0.05)
    rand_imgs = (
        tf.random.uniform((n_new, 32, 32, 1)) * 2 - 1
    )
    old_imgs = tf.concat(
        random.choices(self.examples, k=128-n_new), axis=0
    ) # 나머지는 기존 버퍼로부터 랜덤하게 추출
    inp_imgs = tf.concat([rand_imgs, old_imgs], axis=0)
    inp_imgs = generate_samples(
        self.model, inp_imgs, steps=steps, step_size=step_size, noise = noise
    ) # 샘플을 합치고 랑주뱅 샘플링 실행
    # 만들어진 샘플을 버퍼에 추가하고 최대 샘플 개수를 8192개로 제한
    self.examples = tf.split(inp_imgs, 128, axis = 0) + self.examples
    self.examples = self.examples[:8192]
    return inp_imgs
```

Contrastive Divergence를 통한 훈련

- 아래 그림은 한 스텝을 보여주는 그림
- 각 스텝이 종료된 후 점수 정규화를 고려하지 않음
- 알고리즘에 맞춰 실제 샘플 점수는 내려가고 가짜 샘플 점수는 올라감



Contrastive Divergence를 통한 훈련

- 예제 코드

```
class EBM(models.Model):
    def __init__(self):
        super(EBM, self).__init__()
        self.model = model
        self.buffer = Buffer(self.model)
        self.alpha = ALPHA
        self.loss_metric = metrics.Mean(name="loss")
        self.reg_loss_metric = metrics.Mean(name="reg")
        self.cdiv_loss_metric = metrics.Mean(name="cdiv")
        self.real_out_metric = metrics.Mean(name="real")
        self.fake_out_metric = metrics.Mean(name="fake")

    @property
    def metrics(self):
        return [
            self.loss_metric,
            self.reg_loss_metric,
            self.cdiv_loss_metric,
            self.real_out_metric,
            self.fake_out_metric,
        ]
```

Contrastive Divergence를 통한 훈련

```
def train_step(self, real_imgs):
    real_imgs += tf.random.normal(
        shape=tf.shape(real_imgs), mean=0, stddev=NOISE
    ) # 진짜 이미지에 소량의 랜덤 잡음 추가를 통해 모델이 훈련 세트에 과대적합 되는걸 방지
    real_imgs = tf.clip_by_value(real_imgs, -1.0, 1.0)
    fake_imgs = self.buffer.sample_new_exmps(
        steps=STEPS, step_size=STEP_SIZE, noise=NOISE
    ) # 가짜 이미지를 버퍼에서 샘플링 해옴
    inp_imgs = tf.concat([real_imgs, fake_imgs], axis=0)
    with tf.GradientTape() as training_tape:
        # 진짜와 가짜를 모델에 통과시켜 진짜에 대한 점수와 가짜에 대한 점수를 각각 얻음
        real_out, fake_out = tf.split(self.model(inp_imgs), 2, axis=0)
        cdiv_loss = tf.reduce_mean(fake_out, axis=0) - tf.reduce_mean(
            real_out, axis=0
        )
        # Contrastive Divergence의 Loss는 진짜의 점수와 가짜의 점수 차이
        reg_loss = self.alpha * tf.reduce_mean(
            real_out**2 + fake_out**2, axis=0
        )
        # 점수가 너무 커지지 않게 규제 손실을 더함
        loss = cdiv_loss + reg_loss
    # 역전파를 통해 나온 신경망 가중치의 손실함수의 그래디언트 계산
    grads = training_tape.gradient(loss, self.model.trainable_variables)
    self.optimizer.apply_gradients(
        zip(grads, self.model.trainable_variables)
    )
```

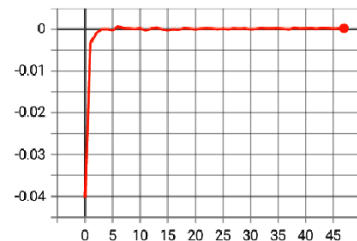
```
self.loss_metric.update_state(loss)
self.reg_loss_metric.update_state(reg_loss)
self.cdiv_loss_metric.update_state(cdiv_loss)
self.real_out_metric.update_state(tf.reduce_mean(real_out, axis=0))
self.fake_out_metric.update_state(tf.reduce_mean(fake_out, axis=0))
return {m.name: m.result() for m in self.metrics}

# 검증 단계에서 test_step 메서드를 통해 랜덤 잡음, 훈련 세트간 Contrastive Divergencer계산
# 모델이 잘 훈련되고 있는지 측정하는데 사용 가능
def test_step(self, real_imgs):
    batch_size = real_imgs.shape[0]
    fake_imgs = (
        tf.random.uniform((batch_size, 32, 32, 1))
        * 2
        - 1
    )
    inp_imgs = tf.concat([real_imgs, fake_imgs], axis=0)
    real_out, fake_out = tf.split(self.model(inp_imgs), 2, axis=0)
    cdiv = tf.reduce_mean(fake_out, axis=0) - tf.reduce_mean(
        real_out, axis=0
    )
    self.cdiv_loss_metric.update_state(cdiv)
    self.real_out_metric.update_state(tf.reduce_mean(real_out, axis=0))
    self.fake_out_metric.update_state(tf.reduce_mean(fake_out, axis=0))
    return {m.name: m.result() for m in self.metrics[2:]}
```

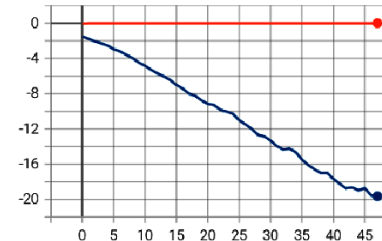
에너지 기반 모델 분석

- 훈련 스텝에서 계산된 손실이 거의 일정하고 변화가 적음
 - 성능이 향상되는 동안 버퍼에 생성된 이미지 품질도 개선되므로 훈련 손실이 크게 떨어지긴 어려움
- 버퍼에서 샘플을 추출하지 않고 랜덤 잡음으로 만든 샘플의 점수를 훈련 세트와 비교하여 판단
 - 대조 발산이 시간에 따라 감소하고 있음
 - 진짜 이미지와 랜덤 잡음을 구별하는 능력이 향상되고 있음
 - 모델 성능이 향상되고 있음을 의미함

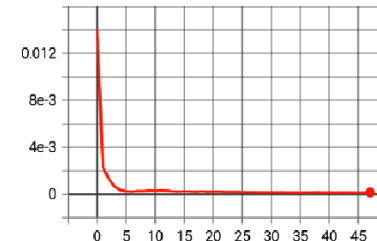
epoch_loss
tag: epoch_loss



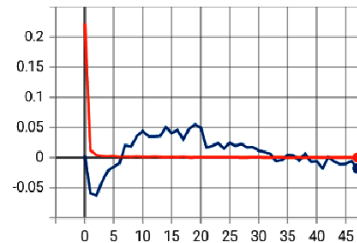
epoch_cdiv
tag: epoch_cdiv



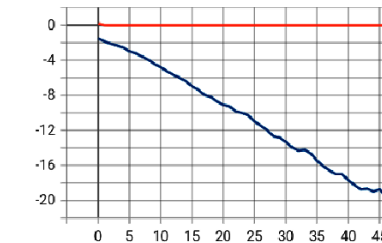
epoch_reg
tag: epoch_reg



epoch_real
tag: epoch_real



epoch_fake
tag: epoch_fake



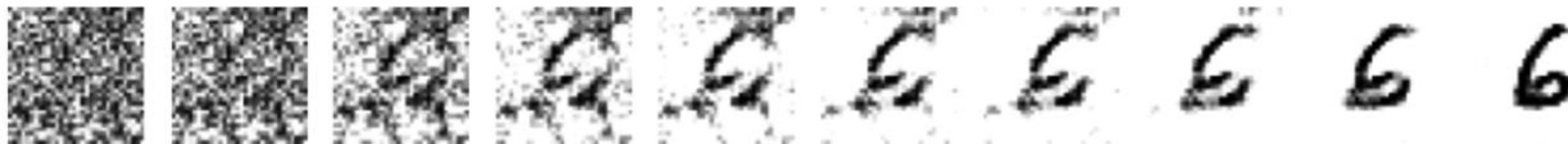
☑ ○ logs/train
☑ ○ logs/validation

EBM을 통해 새로운 샘플 생성

- 새로운 샘플 생성을 위해 Langevin 샘플링을 시작 상태(랜덤 잡음)에서 여러 스텝에 걸쳐서 실행
- 입력에 대한 점수 함수의 그래디언트를 따라 샘플이 아래로 이동함
 - 그럴듯한 샘플이 생성됨
 - 최하단의 그림은 샘플링 과정의 스냅샷

```
start_imgs = (np.random.uniform(size=(10, 32, 32, 1)) * 2 - 1)
gen_img = generate_samples(
    ebm.model,
    start_imgs,
    steps=1000,
    step_size=STEP_SIZE,
    noise=NOISE,
    return_img_per_step=True,
)
```

4 1 6 9 3 8 4 8 0 4



Step 0 1 3 5 10 30 50 100 300 999



기타 에너지 기반 모델

- 초기 EBM 모델은 랑주뱅 샘플링을 사용하지 않았음
- Boltzmann machine을 사용함: $E_{\theta}(v, h) = -\frac{1}{2}(v^T L v + h^T J h + v^T W h)$
 - L, J, W: 모델에 의해 학습되는 가중치(행렬), v: visible unit(픽셀), h: hidden unit(특징)
 - $v^T L v$: visible 끼리도 서로 영향 (픽셀 간 연결)
 - $h^T J h$: hidden 끼리도 서로 영향 (특징 간 연결)
 - $v^T W h$: visible-hidden 간 영향 (픽셀 패턴에 따른 hidden 상태 여부 연결)
 - $-\frac{1}{2}$: 같은 상호작용을 두 번 세는 걸 회피 (가중치 행렬이 대칭이기 때문)
 - 훈련은 Contrastive Divergence를 통해 이루어짐
 - 균형점을 찾을 때 까지 Gibbs 샘플링을 수행함
 - Gibbs 샘플링: 둘 이상의 확률변수의 결합 확률 분포로부터 일련의 표본을 생성함



Gibbs 샘플링

- 앞의 식에서 v, h 를 한 번에 뽑기 어려움
 - $h \sim p(h | v), v \sim p(v | h)$ 를 번갈아 반복하여 (v, h) 가 목표 분포에 가까워지게 만듦
- 볼츠만 머신은 v 끼리 h 끼리도 연결되어 있기 때문에 조건부 분포가 깔끔하게 떨어지지 않음
 - 업데이트가 느려져서 훈련속도도 매우 느리고 hidden unit의 개수를 많이 늘릴 수 없음



Restricted Boltzmann machine

- 같은 종류의 유닛끼리 연결을 제거하여 층을 두개로 나눈 이분 그래프를 생성함
- 이를 통해 RBM을 쌓아서 심층 신뢰 신경망(deep belief network)을 만들 수 있음
- 그럼에도 혼합 시간이 긴 깃스 샘플링이 여전히 필요함
- 그래서 Langevin Dynamics를 활용한 샘플링 방법이 발견되어 선호됨
- 이후 score matching 기법으로 발전함
 - 스코어($\nabla_{\theta} \log p_{\theta}$)를 직접 맞추는 방식
- 결과적으로 Denoising Diffusion Probablistic Model로 발전함
 - 잡음 제거 확산 확률 모델. Chapter 8에서 다룸
 - DALL•E 2와 Imagen과 같은 생성모델을 구현하는데 사용 됨



Summary

- 에너지 기반 모델은 에너지 점수 함수를 사용하는 생성 모델의 한 종류임
 - 이 함수는 실제 샘플에는 낮은 점수, 생성된 샘플에는 높은 점수를 출력하게 훈련된 신경망임
 - 이 점수 함수를 통해 주어진 확률 분포 계산을 위해선 어려운 정규화 분모를 계산해야 함
 - 에너지 기반 모델은 신경망 훈련을 위한 대조 발산과 샘플 생성을 위한 Langevin Dynamics를 활용하여 이 문제를 해결함
- 에너지 함수는 생성된 샘플 점수와 훈련 데이터 점수의 차이를 최소화 함으로써 훈련 됨
 - 대조 발산이라고 하며, 까다로운 정규화 분모 계산이 불필요함
 - 효율성 유지를 위해 가짜 샘플에 대한 샘플링 과정을 근사화 함
- 심층 에너지 기반 모델은 Langevin Dynamics를 통해 이루어짐
 - 입력 이미지에 대한 점수의 기울기를 사용하여 기울기가 감소하는 방향으로 입력을 조금씩 업데이트 하며 그럴듯한 샘플로 점차 변환됨
 - 이 방식은 Restricted Boltzmann Machine에서 사용하는 Gibbs 샘플링과 같은 방법을 개선한 방식임

